

src\case_studies\F_vtol\dynamics.py

Generated: 2026-02-26 12:33:41

dynamics.py

src\case_studies\F_vtol\dynamics.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from case_studies.F_vtol import eom_generated
from ..common.dynamics_base import DynamicsBase
from . import params as P

class VTOLDynamics(DynamicsBase):
    def __init__(self, alpha=0.0, F_wind=0.0):
        super().__init__(
            state0=np.array([P.z0, P.h0, P.theta0, P.zdot0, P.hdot0, P.thetadot0]),
            u_max=P.force_max,
            u_min=-P.force_max,
            dt=P.ts,
        )
        self.mc = self.randomize_parameter(P.mc, alpha)
        self.mr = self.randomize_parameter(P.mr, alpha) # TODO: noise or not?
        self.Jc = self.randomize_parameter(P.Jc, alpha)
        self.d = self.randomize_parameter(P.d, alpha)
        self.mu = self.randomize_parameter(P.mu, alpha)
        self.F_wind = F_wind
        self.g = P.g

    def f(self, x, u):
        # could either use the generated equations of motion from
        # eom_generated.py, or re-derive them here directly. Here, I will
        # re-derive them directly for clarity. But if we wanted to use the
        # generated equations, we would do something like:
        #
        # xdot = eom_generated.calculate_eom(x, u, self.mc, self.mr, self.Jc, self.d, sel

        z, h, theta, zdot, hdot, thetadot = x.flatten()
        fr, fl = u.flatten()
        F = fr + fl
        tau = P.d * (fr - fl)

        mass = self.mc + 2 * self.mr
        thrust_lat = -F * np.sin(theta)
        thrust_lon = F * np.cos(theta)
        drag_lat = -self.mu * zdot
        weight = mass * self.g
```

```
inertia = self.Jc + 2 * self.mr * self.d**2

zddot = (thrust_lat + drag_lat + self.F_wind) / mass
hddot = (thrust_lon - weight) / mass
thetaddot = tau / inertia

xdot = np.array([zdot, hdot, thetadot, zddot, hddot, thetaddot])
return xdot

def h(self):
    z, h, theta = self.state[:3]
    y = np.array([z, h, theta])
    return y
```